

<https://drive.google.com/file/d/1XitrZWfVlgj0dj3Z1LYlvekoVH5U3I6M/view>

https://drive.google.com/file/d/1A2He5fzqdv_RAzgL-ilyDc93yabQPxQj/view



Logistic Regression



Logistic Regression

- **supervised** machine learning algorithm
- used for **classification problems**, especially when the output has only **two possible outcomes** (**binary classification**)

So, the **dependent variable** (Y) is **binary (dichotomous)** $Y \in \{0, 1\}$
> we predict the dependent variable!!

- In **linear regression**, we **predict** the **value of Y directly**.
- Problem: The predicted value can be:
 - *Less than 0* ✗
 - *Greater than 1* ✗
- But probabilities must always lie between: $0 \leq P(Y) \leq 1$

To solve this issue, Logistic Regression **predicts** a **probability** using a special function called the **Logistic (Sigmoid) Function**



Logistic (Sigmoid) Function

The logistic function converts any value into a **probability between 0 and 1**.

$$P(Y) = \frac{1}{1 + e^{-z}}$$

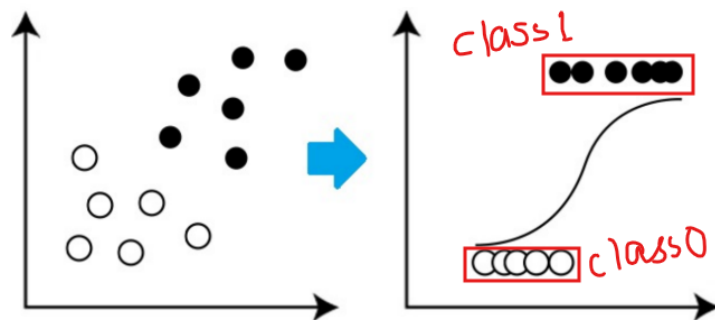
where

$$z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_k X_k$$

› Properties of Sigmoid Function

- **Output** always between **0 and 1**
- Produces an **S-shaped curve**
- Used to **estimate probabilities**

Sigmoid Curve Interpretation



› Instead of drawing a straight line like Linear Regression, Logistic Regression fits an **S-shaped curve**.

x Value	Probability
Very Small	Close to 0
Medium	Around 0.5
Very Large	Close to 1

What is the "Regression" Component?

Regression means **finding the relationship between**:

- Predictor variables (X)
- Outcome variable (Y)

Logistic Regression:

- Measures association between variables
 - **Predicts probabilities**
 - Builds classification models
-



The Logistic Regression Model

The logistic regression equation is:

$$\ln \left(\frac{P(Y)}{1 - P(Y)} \right) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_k X_k$$

Outcome Variable (Y)

Y is binary, **Dichotomous Outcome** $\Rightarrow$ only two possible categories.

- NO $\Rightarrow$ 0
- YES $\Rightarrow$ 1

Examples:

- Fail = 0, Pass = 1
- Healthy = 0, Sick = 1



Predictor Variables

- The **X values** are the **input features**. $X_1, X_2, \dots, X_k$
- These variables help predict Y.



Intercept (β_0)

- Also called: **Bias** OR **Constant term**
- the **starting value** of the model **when all X values are zero**.



Coefficients ($\beta_1, \beta_2, \dots, \beta_k$)

- **How much each predictor affects the outcome.**

> Interpretation

If β is:

✓ **Positive** → **increases probability**

✗ **Negative** → **decreases probability**



Odds and Log Odds

Probability of success: $P(Y)$

Example: $P(Y)=0.8 \Rightarrow 80\%$ chance of success.



Odds > **compare success against failure.**

$$Odds = \frac{P(Y)}{1 - P(Y)}$$

$$Odds = \frac{0.8}{0.2} = 4$$

Success is 4 times more likely than failure.

Log Odds (Logit)

Taking natural log of odds:

$$Logit(P) = \ln \left(\frac{P(Y)}{1 - P(Y)} \right)$$

This is exactly what Logistic Regression models.

Why Model Log Odds Instead of Probability?

Because probabilities → **restricted between 0 and 1**

But **log odds**: $-\infty$ to $+\infty$, This **allows** a **linear equation to work**.

Converting Log Odds Back to Probability

Starting from:

$$\ln \left(\frac{P(Y)}{1 - P(Y)} \right) = \beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k$$

Let

$$z = \beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k$$

Then:

$$P(Y) = \frac{e^z}{1 + e^z}$$

or equivalently:

$$P(Y) = \frac{1}{1 + e^{-z}}$$

^ This is the **final prediction equation**.

Feature	Linear Regression	Logistic Regression
Output	Continuous value	Probability
Used For	Prediction	Classification
Range	$-\infty$ to $+\infty$	0 to 1
Equation	$Y = \beta_0 + \beta_1 X + \varepsilon$	$\text{logit}(P) = \beta_0 + \beta_1 X$
Curve	Straight line	S-curve
Target Variable	Continuous	Binary

Commonality Between Linear and Logistic Regression

Both models:

-  **Use predictors** ($X_1, X_2, \dots, X_k$)
-  **Learn coefficients** β
-  **Have an intercept** β_0
-  **Find relationships between variables**
-  **Belong to the **Generalized Linear Model (GLM)** family**

> Generalized Linear Model **extends linear regression** to **different types of outputs**.

Logistic Regression Training Algorithm

The model **learns by repeatedly adjusting weights.**

- Calculate the **prediction $\hat{y}$ using the current parameters (W and b)**
- Calculate the **loss of the current values**
- Calculate the **gradients of the loss function** with respect to the parameters
- **Adjust** the **weights** (optimize) using the gradients
- **Repeat** for the number of epochs, i.e. the number of times to go through the provided examples (dataset)

Step 1: Initialize Parameters

Randomly choose:

- Weights (W)
- Bias (b)

Step 2: Calculate Prediction

Compute: $z = WX + b$

Apply sigmoid:

$$\hat{y} = \frac{1}{1 + e^{-z}}$$

> gives **predicted probability.**

Step 3: Calculate Loss

Measure prediction error.

› Binary Cross-Entropy Loss

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Step 4: Calculate Gradients

Find:

$$\frac{\partial L}{\partial W} \quad \text{and} \quad \frac{\partial L}{\partial b}$$

These tell us [how much to adjust parameters](#).

Step 5: Update Weights

Gradient Descent:

$$W = W - \alpha \frac{\partial L}{\partial W} \quad b = b - \alpha \frac{\partial L}{\partial b}$$

Alpha = learning rate

Step 6: Repeat

Repeat Steps 2–5 for many iterations (epochs).

Each iteration:

- Loss decreases
- Predictions improve
- Model learns patterns

Logistic Regression as a Neural Network

Logistic Regression

$$z = b + a_1x_1 + a_2x_2 + a_3x_3$$
$$p = 1.0 / (1.0 + e^{-z})$$

Ex:

$$\begin{array}{ll} x_1 = 1.0 & a_1 = 0.01 \\ x_2 = 2.0 & a_2 = 0.02 \\ x_3 = 3.0 & a_3 = 0.03 \\ & b = 0.05 \end{array}$$

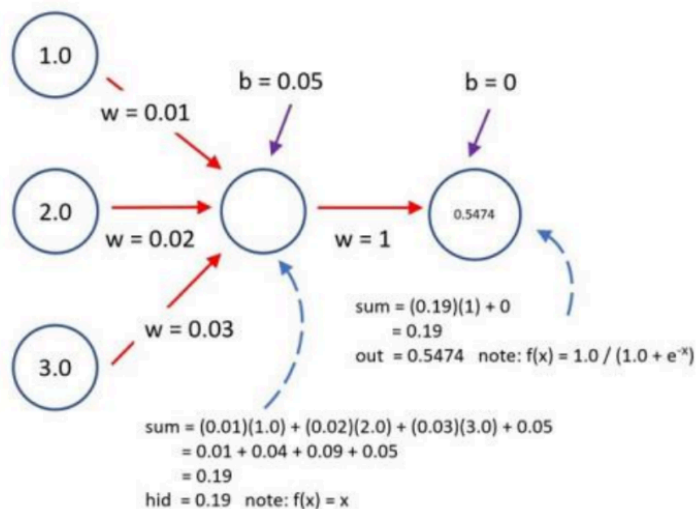
$$\begin{aligned} z &= (0.05) + (0.01)(1.0) + \\ &\quad (0.02)(2.0) + (0.03)(3.0) \\ &= 0.05 + 0.01 + 0.04 + 0.09 \\ &= 0.19 \end{aligned}$$

$$p = 1.0 / (1.0 + e^{-0.19})$$
$$= 0.5474 \text{ (predicted class = 1)}$$

Neural Network

single hidden layer, identity activation $f(x) = x$

single output node, logistic sigmoid activation $f(x) = 1 / (1 + e^{-x})$



Loss Function

A **Loss Function** tells us **how much error the model made** for one training example.

- Small loss for **good predictions**
- Large loss for **bad predictions**

✗ Why Not Use Squared Error?

In Linear Regression we use: $L = (y - \hat{y})^2$

Problem

For Logistic Regression: Using squared error with the sigmoid function creates a:
Non-Convex Cost Function

Meaning:

- **Many local minimums**
- **Difficult optimization**
- **Gradient Descent may get stuck**

✗ Hard to find best solution (global optimum) & to minimize cost value

🎯 Solution: **Cross Entropy Loss**

Logistic Regression uses: **Cross Entropy Loss**

Also called:

- **Log Loss**
- **Logistic Loss**

This gives us a:

- ☒ Convex optimization problem
- ☒ Easier training
- ☒ Better convergence

$$\text{If } y = 1: \quad p(y|x) = \hat{y}$$

$$\text{If } y = 0: \quad p(y|x) = 1 - \hat{y}$$

$$P(y|x) = \hat{y}^y (1-\hat{y})^{(1-y)}$$

$$\log(P(y|x)) = \log \hat{y}^y (1-\hat{y})^{(1-y)}$$

$$= y \log \hat{y} + (1-y) \log(1-\hat{y})$$

What is Log Likelihood?

Likelihood $\Rightarrow$ "How likely are the observed data given the current model?"

We want this value to be: **As Large As Possible**

Goal $\triangleright$ **Maximize Log Likelihood**

$\triangleright$ Machine Learning algorithms usually: **Minimize** instead of maximize.

So we simply multiply by: -1



Cross Entropy Loss

The loss for one training example becomes:

$$L_{CE} = - \left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}) \right]$$

Cross Entropy:

- **Rewards correct predictions**
 - **Punishes** wrong predictions **heavily**
-



Loss for Entire Dataset

For one example:

$$L_{CE} = - \left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}) \right]$$

For m training examples:

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)})$$

Substituting the loss:

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

This is called the **Cost Function**



Goal of Training

Find: w and b such that **J(w,b)** becomes **as small as possible**.

Gradient Descent

- Used to **minimize $J(w,b)$**

Gradient Descent Algorithm [**same algo as above**]

> Step 1

Initialize:

- Weights (w)
- Bias (b)

with random values.

> Step 2

Compute prediction: $\hat{y} = \sigma(w^T x + b)$

> Step 3

Compute cost: $J(w,b)$

> Step 4

Compute gradients

$$\frac{\partial J}{\partial w} \quad \& \quad \frac{\partial J}{\partial b}$$

› Step 5

$$w = w - \alpha \frac{\partial J}{\partial w} \quad b = b - \alpha \frac{\partial J}{\partial b}$$

Alpha $\Rightarrow$ learning rate

› Step 6

Repeat Steps 2–5 for many iterations.

Each iteration:

-  Cost decreases
 -  Predictions improve
 -  Parameters become optimal
-

Logistic Regression

- Used for [binary classification](#).
- Predicts [probabilities between 0 and 1](#).
- Uses the [sigmoid \(logistic\) function](#).
- [Models the log odds \(logit\)](#) of the outcome.
- [Output variable is dichotomous \(0 or 1\)](#).
- β_0 is intercept.
- $\beta_1, \beta_2, \dots, \beta_n$ are coefficients.
- Belongs to the [Generalized Linear Model \(GLM\)](#) family.
- [Trained using gradient descent](#) and [cross-entropy loss](#).
- Final prediction is obtained using a threshold (usually 0.5).

Loss Function

- Measures prediction error.
- Squared error is not preferred in Logistic Regression.

- Squared error creates a non-convex optimization problem.
- Logistic Regression uses Cross Entropy Loss.
- Cross Entropy is derived from Log Likelihood.
- Log Likelihood is maximized.
- Cross Entropy Loss is minimized.

example:

- ▶ Calculate the probability of pass students who studied 33 hours.
- ▶ At least how many hours student should study that makes he will pass the course with the probability of more than 95%.
- ▶ Given the model suggested by the optimizer for odds of passing the course is,

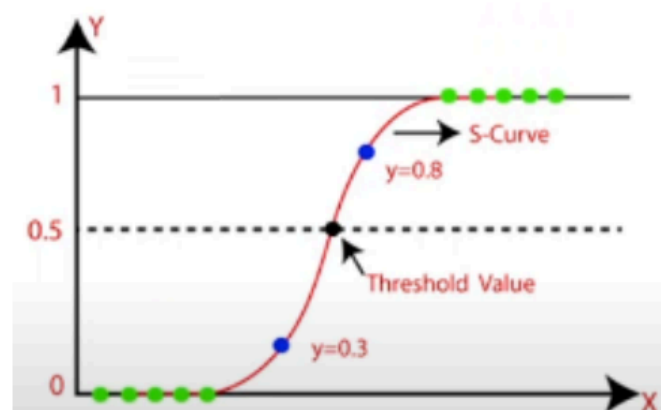
$$\text{Log(odds)} = -64 + 2 * \text{hours}$$

Hours Study	Pass (1) / Fail (0)
29	0
15	0
33	1
28	1
39	1

Iqbal Farooq

- ▶ We use Sigmoid Function in logistic regression

$$s(x) = \frac{1}{1 + e^{-x}}$$



- Calculate the probability of pass students who studied 33 hours.

$$\log(\text{odds}) = z = -64 + 2 * \text{hours}$$

- So, hours= 33 which make the equation like

$$\begin{aligned} \text{Log(odds)} &= z = -64 + 2 * 33 \\ Z &= 2 \end{aligned}$$

$$s(x) = \frac{1}{1+e^{-x}}$$

By putting the value to find the probability, we get 0.88

$$p = \frac{1}{1+e^{-z}}$$

That is 88% chances student will pass the exam if he study 33%

- At least how many hours student should study that makes he will pass the course with the probability of more than 95%.

$$p = \frac{1}{1+e^{-z}} = 0.95 \quad 2??$$

$$0.95 * (1 + e^{-z}) = 1$$

$$0.95 * e^{-z} = 1 - 0.95$$

$$e^{-z} = \frac{0.05}{0.95} = 0.0526$$

$$\ln(e^{-z}) = \ln(0.0526)$$

$$\ln(e^x) = x$$

$$-z = \ln(0.0526) = -2.94$$

$$z = 2.94$$

$$\log(\text{odds}) = z = -64 + 2 * \text{hours}$$

$$2.94 = -64 + 2 * \text{hours}$$

$$2 * \text{hours} = 2.94 + 64$$

$$2 * \text{hours} = 66.94$$

$$\text{hours} = \frac{66.94}{2}$$

$$\text{hours} = 33.47 \text{ Hours}$$